

# SawaFlix Reels Data Pipeline & Performance Architecture

**Date:** July 2026 **Document Type:** Technical Architecture & Implementation Notes **Status:** Deployed

## Summary

This document outlines the architectural changes implemented to optimize the SawaFlix "Reels" content feed. The core objective of this update was to solve three critical issues with the previous system:

1. **Content Relevancy:** Keyword-based YouTube API searches were returning generic, irrelevant results instead of authentic Cameroon cultural content.
2. **Performance Bottlenecks:** Querying third-party APIs on client load caused significant UI blocking and high initial load times.
3. **API Rate Limiting:** Live requests scaled linearly with user traffic, risking rapid consumption of YouTube Data API quotas.

The new architecture decouples content fetching from the client render lifecycle by introducing an asynchronous backend cron pipeline, a centralized Supabase storage layer, and an aggressive frontend caching strategy using IndexedDB.

## 2. Architecture Overview

The system is now divided into two distinct lifecycles: **Data Ingestion (Backend)** and **Data Delivery (Frontend)**.

### 2.1 Data Ingestion (The Cron Pipeline)

Instead of relying on dynamic user searches, we transitioned to a "curated walled-garden" approach.

- **Target Channels:** We manually verified a specific list of YouTube channels known for producing high-quality Cameroon cultural content.
- **Cron Job:** A Node.js background worker (`sawaflix-backend/src/controllers/cron.controller.ts`) is scheduled to execute every 6 hours.
- **Execution Flow:**
  1. The worker pings the YouTube Data API exclusively for recent uploads from the curated channel list.
  2. The payload is cleaned, deduplicated, and securely written to our Supabase database (`curated_videos` table).
  3. This guarantees that at the end of every day, a pool of at least 100 highly relevant, unique videos is available to the application.

## 2.2 Data Delivery & Caching (The Client)

The SawaFlix frontend (Next.js) now reads exclusively from our internal API, eliminating external dependencies during the critical rendering path.

**The `useVideos` Hook Implementation:** We implemented a strict *Stale-While-Revalidate* cache pattern using the browser's native IndexedDB (`idb-keyval`).

1. **Instant Render:** On component mount, the hook immediately reads from IndexedDB and populates the UI state in milliseconds.
2. **Background Sync:** The hook fires an asynchronous request to the SawaFlix backend.
3. **Cache Invalidation:** Once the backend returns fresh data from Supabase, the hook overwrites the IndexedDB cache and silently updates the UI.
4. **Resiliency:** If the backend or Supabase is unreachable (e.g., local network timeouts), the application gracefully degrades and continues serving the cached feed without throwing user-facing errors.

### 3. Feed Shuffling Logic

To mimic the dynamic nature of short-form video platforms (like TikTok or Instagram Reels), it is crucial that users do not see the exact same sequential feed on every application launch.

Since the backend API returns videos sorted chronologically by `published_at` (to ensure the feed is seeded with the newest content), we implemented a deterministic shuffle on the client side.

- **Implementation:** Before setting the React state or committing to IndexedDB, the incoming video array is randomized using a highly efficient sorting algorithm (`.sort(() => Math.random() - 0.5)`).
- **Outcome:** Even if the cron job hasn't pulled new videos in the last hour, refreshing the browser guarantees a fresh permutation of the curated content.

### 4. Security & Maintenance Considerations

- **API Keys:** YouTube API keys and Supabase Service Role keys are now strictly isolated to the backend environment variables, entirely removed from the Next.js client bundle.
- **Scalability:** By shifting to a database-first approach, frontend read volume scales independently of YouTube API quota limits. We can scale to millions of page views while making exactly 4 YouTube API calls per day.